CURSO PARA PRINCIPIANTES · NO SE NECESITA NINGÚN CONOCIMIENTO PREVIO

# Crear una web de la A a la Z

Entender cómo se construye, se publica y se gestiona a diario una web real — abriendo, archivo por archivo, la de **The Evergreen Project**.

Web: dimogdor.github.io/Evergreen · Código fuente: github.com/Dimogdor/Evergreen

**CÓMO LEER ESTE CURSO**

Cada capítulo explica una idea de forma sencilla, luego muestra **dónde verla en la web** (recuadro verde «Mirar») y a veces propone una pequeña **prueba** (recuadro naranja). Los recuadros azules «Para decir» son frases clave para la explicación en voz alta. No hace falta recordarlo todo: el objetivo es entender *quién hace qué*.

# Índice


1. Cómo funciona una web
2. Los tres lenguajes de la web: HTML, CSS, JavaScript
3. La caja de herramientas
4. El repositorio de GitHub: la carpeta del proyecto
5. Astro: el «generador» de la web
6. Separar el contenido del código (YAML)
7. El diseño en la práctica: de la idea a la pantalla
8. Maquetación y diseño adaptable (móvil, tableta, ordenador)
9. Fotos y vídeos
10. Una web en 8 idiomas
11. Las funcionalidades, una a una
12. El panel de administración (sin código)
13. El pequeño servidor: el Worker
14. Aparecer en Google (SEO)
15. Accesibilidad: una web para todo el mundo
16. Seguridad
17. Rapidez (rendimiento)
18. Probar la web
19. Publicar: alojamiento, dominio, DNS
20. El ciclo de vida de un cambio
21. Por dónde empezar a aprender
22. Glosario


**PARA DECIR COMO INTRODUCCIÓN**

«Una web no es más que archivos de texto que un navegador sabe leer. Todo lo demás — las herramientas, los servicios — sirve para escribir esos archivos más rápido, hacerlos bonitos y publicarlos.»

# 1 · Cómo funciona una web

Cuando escribes una dirección en un navegador (Chrome, Safari, Firefox…), pasan tres cosas:

Navegador → **pide** → Servidor (un ordenador siempre encendido) → **devuelve archivos** → El navegador los muestra

1. **La dirección** (URL), p. ej. `https://dimogdor.github.io/Evergreen/es/contact/`, indica a qué servidor llamar y qué página pedir.
2. **El servidor** devuelve archivos: una página **HTML**, hojas de estilo **CSS**, scripts **JavaScript**, imágenes.
3. **El navegador** lo junta todo y muestra la página.

## ¿Web estática o dinámica?

| | Web estática (nuestro caso) | Web dinámica (p. ej. WordPress) |
|---|---|---|
| Principio | Las páginas se fabrican **por adelantado**, una vez, y luego simplemente se envían. | Cada página se fabrica **en cada visita** con un programa y una base de datos. |
| Velocidad | Muy rápida | Más lenta |
| Seguridad | Muy pocos puntos débiles posibles | Actualizaciones de seguridad frecuentes |
| Coste | Alojamiento gratuito | Normalmente un servidor de pago |

La web de Evergreen es **estática**: después de cada cambio, una herramienta fabrica todas las páginas (¡171!) en unos segundos y luego se dejan en el alojamiento. Más adelante veremos cómo tenemos, aun así, un panel de administración y formularios.

> **MIRAR**
>
> Abre la web y haz clic derecho → **«Ver código fuente de la página»**: es el HTML que ha enviado el servidor. Después clic derecho → **«Inspeccionar»**: son las **herramientas para desarrolladores** del navegador, el mejor amigo de quien crea webs.

# 2 · Los tres lenguajes de la web

Todas las webs del mundo se basan en tres lenguajes, cada uno con un papel preciso. Una imagen sencilla: una casa.

| Lenguaje | Papel | En la casa |
|---|---|---|
| **HTML** | La **estructura** y el contenido: títulos, párrafos, imágenes, enlaces, formularios. | Las paredes y las habitaciones |
| **CSS** | El **aspecto**: colores, tipografías, tamaños, espacios, disposición. | La pintura y la decoración |
| **JavaScript** | El **comportamiento**: lo que reacciona al hacer clic, lo que cambia sin recargar. | La electricidad, los interruptores |

## HTML: «etiquetas»

```
<h1>Contacto</h1>                    ← un título grande
<p>Escríbenos, respondemos rápido.</p>  ← un párrafo
<a href="mailto:evergreenecovillage@gmail.com">Escríbenos</a>  ← un enlace
<img src="tienda.webp" alt="Una tienda en el bosque">        ← una imagen + su descripción
```

## CSS: «reglas»

```
h1 { font-family: 'EB Garamond'; color: #22402a; }   ← todos los títulos grandes en verde oscuro
.btn { border-radius: 999px; padding: 0.85em 1.4em; } ← botones redondeados
```

## JavaScript: «instrucciones»

```
boton.addEventListener('click', () => {                ← al hacer clic en el botón…
  navigator.clipboard.writeText('GR7701103910000039100269800');  ← …copiar el IBAN
});
```

**MIRAR EN EL REPOSITORIO**

- CSS: `src/styles/global.css` — todo el estilo de la web en un solo archivo.
- JavaScript: al final de `src/pages/[lang]/[page].astro`, el botón «Copiar» del IBAN (las líneas `<script>`).
- HTML: en todos los archivos `.astro` (ver capítulo 5).

## ¿Y TypeScript?

Es JavaScript con «etiquetas» que precisan el tipo de cada dato (texto, número…). Evita errores. Los archivos `.ts` del proyecto (p. ej. `src/calendar.ts`) están escritos así.

**PARA DECIR**

«Empieza por HTML y CSS: en dos semanas puedes hacer una página de verdad. JavaScript viene después, cuando quieres que la página reaccione.»

# 3 · La caja de herramientas

| Herramienta | Para qué sirve | ¿Gratis? |
|---|---|---|
| **Un editor de código**: VS Code | Escribir y organizar los archivos, con colores y sugerencias. | Sí |
| **Un navegador** + sus herramientas para desarrolladores (F12) | Ver el resultado, inspeccionar, probar en un «falso» móvil. | Sí |
| **La terminal** | Escribir comandos (arrancar la web, instalar herramientas). | Sí |
| **Node.js** y **npm** | Node ejecuta JavaScript en el ordenador; npm instala «paquetes» (herramientas escritas por otras personas). | Sí |
| **Git** | Guardar el historial de cada cambio (como «Deshacer», pero para siempre). | Sí |
| **GitHub** | Guardar el proyecto en línea, colaborar, lanzar la publicación automática. | Sí |
| **Figma** (opcional) | Dibujar maquetas antes de programar. | Sí (plan básico) |

## Los tres comandos del proyecto

```
npm install     ← descarga las herramientas de package.json (una sola vez)
npm run dev     ← arranca la web en el ordenador: http://localhost:4321 (se actualiza al guardar)
npm run build   ← fabrica la versión final en la carpeta dist/
```

**MIRAR**

`package.json`: el «carné de identidad» del proyecto. Muestra los comandos (`scripts`) y los paquetes usados (`dependencies`): **astro** (el generador), **@astrojs/sitemap** (mapa del sitio para Google), **marked** (convierte texto simple en HTML), **yaml** (lee los archivos de contenido) y dos tipografías (**Inter** y **EB Garamond**).
`.node-version`: la versión de Node que se usa (22).

## Git en 5 palabras

- **Repositorio**: la carpeta del proyecto con todo su historial.
- **Commit**: una «foto» de los archivos en un momento, con un mensaje («Inhabitants: new photo of our dogs»).
- **Rama** (branch): una copia de trabajo paralela para probar sin romper la versión oficial (`main`).
- **Pull request**: la petición de fusionar una rama en `main`, con revisión.
- **Push / pull**: enviar / traer los commits a / desde GitHub.

**PRUEBA**

En github.com/Dimogdor/Evergreen, haz clic en **«Commits»**: ves cada cambio hecho en la web, quién lo hizo y cuándo, y puedes mostrar exactamente las líneas modificadas.

# 4 · El repositorio: el plano del proyecto

Así están organizadas las carpetas. Ordenar bien es la mitad del trabajo.

```
Evergreen/
├── package.json            ← carné de identidad, comandos, herramientas
├── astro.config.mjs        ← ajustes del generador (dirección de la web, seguridad)
├── README.md               ← manual del proyecto
├── docs/GUIDE-ADMIN.md     ← manual para las personas administradoras
├── public/                 ← archivos copiados tal cual
│   ├── admin/              ← el panel de administración (config.yml, index.html)
│   ├── uploads/videos/     ← vídeos
│   └── _headers            ← reglas de seguridad/caché para el alojamiento
├── src/                    ← el corazón de la web
│   ├── pages/              ← una página = un archivo (capítulo 5)
│   ├── layouts/Base.astro  ← el marco común: cabecera, menú, pie de página
│   ├── components/         ← piezas reutilizables (imagen, lista de eventos…)
│   ├── styles/global.css   ← todo el diseño
│   ├── content/            ← TODO el texto de la web, en archivos YAML (capítulo 6)
│   ├── assets/uploads/     ← fotos originales (optimizadas automáticamente)
│   ├── lib.ts              ← pequeñas funciones útiles (leer contenido, enlaces…)
│   ├── i18n.ts             ← los idiomas
│   └── calendar.ts         ← fechas, husos horarios, archivos de calendario
├── worker/                 ← el pequeño servidor (acceso admin, historias, ayuda a la escritura)
├── tests/                  ← comprobaciones automáticas
└── .github/workflows/      ← robots de GitHub: publicación, compresión de vídeo
```

**PARA DECIR**

«El código (páginas, diseño) y el contenido (textos, fotos) están en carpetas separadas. Quienes llevan el lugar solo tocan el contenido, desde el panel; el código casi nunca cambia.»

# 5 · Astro: el «generador» de la web

Escribir 171 páginas HTML a mano sería imposible (8 idiomas × una veintena de páginas). Por eso usamos un **generador de webs estáticas**: escribimos **una plantilla** por tipo de página y la herramienta la rellena con cada contenido, en cada idioma. Aquí es **Astro** (astro.build).

## Un archivo .astro = dos partes

```
---
// Parte 1, entre los guiones: JavaScript que prepara los datos (se ejecuta al fabricar la web)
const page = await one('pages', 'contact', lang);
---
<!-- Parte 2: HTML, en el que se insertan los datos entre llaves -->
<h1>{page.title}</h1>
<p class="lead">{page.intro}</p>
```

## Las páginas = las direcciones

En `src/pages/`, el nombre del archivo se convierte en la dirección. Los corchetes `[ ]` significan «variable»:

| Archivo | Direcciones que produce |
|---|---|
| `src/pages/[lang]/index.astro` | /el/, /en/, /es/… (las 8 páginas de inicio) |
| `src/pages/[lang]/[page].astro` | /es/contact/, /es/place/, /es/support/… (todas las páginas de texto) |
| `src/pages/[lang]/activities/[slug].astro` | /es/activities/retreats/… (una página por actividad) |
| `src/pages/[lang]/calendar.astro` | /es/calendar/ |
| `src/pages/[lang]/gallery.astro`, `stories.astro` | la galería, las historias |
| `src/pages/calendar.ics.ts` | /calendar.ics: ¡un archivo de calendario, no una página! |

## El layout: el marco común

`src/layouts/Base.astro` contiene lo que es igual en todas las páginas: el `<head>` invisible (título de la pestaña, descripción para Google, icono), la cabecera con el logo y el menú, el selector de idioma, el pie de página con los contactos. Cada página «encaja» dentro con `<Base> … </Base>`.

## Los componentes: piezas de LEGO

El más sencillo para entender: abre `src/components/Story.astro` (12 líneas) — una foto, una cita (`<blockquote>`), el nombre y el país.

| Componente | Qué hace |
|---|---|
| `src/components/Img.astro` | Muestra cualquier foto, optimizada automáticamente (capítulo 9). |
| `Blocks.astro` | Las secciones «título + texto + foto» que alternan izquierda/derecha. |
| `EventList.astro` | La lista de eventos con fechas, precios y botón «Añadir al calendario». |
| `Story.astro` | Una tarjeta de historia. |
| `Video.astro` | Un vídeo de YouTube/Vimeo que solo se carga al hacer clic. |

# 6 · Separar el contenido del código

Todos los textos, fechas, precios y rutas de fotos están en `src/content/`, en formato **YAML**: un formato de texto muy legible, hecho de líneas «clave: valor» y sangrías.

> **MIRAR: UN EVENTO**
>
> `src/content/events/essence-of-yoga-2026.yml`

```
el:                                  ← la versión griega (idioma principal)
  start: '2026-11-28'                ← fecha de inicio
  end: '2026-12-05'                  ← fecha de fin
  category: retreat
  image: /src/assets/uploads/yin-shala.webp
  link: https://bookretreats.com/r/8-day-essence-of-yoga-…
  title: Essence of Yoga — χειμερινό εντατικό …
  price: 380 € early bird (κράτηση έως 28/10/2026) · 450 € κανονική τιμή
es:                                  ← en español: solo lo que se traduce
  title: Essence of Yoga — inmersión de invierno con Panos Katsandris
  price: 380 € early bird (reserva hasta el 28/10/2026) · 450 € precio normal
```

| Carpeta | Contenido |
|---|---|
| `settings/site.yml` | Nombre, e-mail, teléfono, redes, IBAN, colores de la web |
| `settings/ui.yml` | Los textos pequeños de la interfaz (menú, botones) en los 8 idiomas |
| `pages/` | El texto de cada página (inicio, el lugar, forma de vida…) |
| `programs/` | Las actividades (voluntariado, retiros…) |
| `events/` | El calendario: un archivo por evento |
| `inhabitants/` | Los habitantes: Katerina, Kostas, los perros, los gatos, las gallinas… |
| `albums/`, `testimonials/`, `extra/` | Galería, historias, páginas creadas en el panel |

¿Cómo lee el código este contenido? `src/content.config.ts` declara las carpetas, y la función `list()` de `src/lib.ts` las lee y elige el idioma correcto.

> **PARA DECIR**
>
> «Sin base de datos: el contenido son simples archivos de texto ordenados en carpetas. Se pueden leer, guardar, volver atrás. El panel solo edita estos archivos por nosotros.»

# 7 · El diseño en la práctica: de la idea a la pantalla

## Paso 1 — Reunir inspiración («moodboard»)

Antes de programar, miramos webs que nos gustan y anotamos *por qué*. Para Evergreen: kaleidoscope.media (títulos grandes y elegantes, mucho espacio, secciones numeradas «01 —») y evergreenyogastudios.com (la identidad de la familia Evergreen). También reunimos el material: logo, fotos recientes, tono de los textos.

## Paso 2 — Elegir una paleta

Bastan 4 a 6 colores, sacados del tema: aquí papel crema, musgo, bosque profundo, terracota. Comprobamos el **contraste** (texto legible sobre el fondo) con una herramienta como *webaim.org/resources/contrastchecker*.

```
:root {                       ← src/styles/global.css, arriba del todo
  --paper: #f4f0e6;           fondo crema
  --ink: #17231a;             texto casi negro, con un toque verde
  --moss: #3d673f;            verde musgo
  --moss-deep: #22402a;       verde bosque (banda CES, pie de página)
  --clay: #b0622a;            terracota (toques de acento)
}
```

Estas «variables» (`--paper`…) se reutilizan en todas partes; cambiar una línea cambia toda la web. Incluso se puede cambiar desde el panel (Ajustes → Colores).

## Paso 3 — Elegir dos tipografías

Una **tipografía con serifa** para los títulos (EB Garamond: cálida, «de libro») y una **tipografía sencilla** para el texto (Inter: muy legible en pantalla). Vienen de Google Fonts pero están **instaladas en el proyecto** mediante los paquetes `@fontsource-variable/…`: ninguna llamada a Google, más rápido y más respetuoso con la privacidad.

## Paso 4 — Una escala de espacios y tamaños

En lugar de tamaños fijos usamos `clamp(mín, ideal, máx)`: el título crece con la pantalla sin quedar nunca demasiado pequeño ni demasiado grande. Ejemplo: `--gutter: clamp(16px, 4vw, 56px)` = el margen lateral.

## Paso 5 — «Componentes visuales» coherentes

Botones redondos en mayúsculas espaciadas (`.btn`), pequeñas etiquetas (`.label`), tarjetas foto + título (`.card`), líneas horizontales numeradas (`.rule`). Una vez definidos, se reutilizan: la web parece «diseñada», no improvisada.

## Paso 6 — Maqueta (opcional) y luego código

En Figma dibujamos la página de inicio en versión ordenador y móvil. Luego la traducimos a HTML (estructura) y CSS (estilo) — y ajustamos directamente en el navegador con las herramientas para desarrolladores.

> **PRUEBA**
>
> Abre la web, F12 → pestaña «Elementos», haz clic en `<html>` y cambia `--moss-deep` por `#5a2a6e`: toda la web cambia de color en directo (no se rompe nada, basta con recargar).

# 8 · Maquetación y diseño adaptable

Una misma web debe verse bien en un móvil (390 px de ancho) y en una pantalla grande (1440 px o más). Es el **diseño adaptable** (responsive).

## Las dos herramientas de maquetación del CSS moderno

| Herramienta | Para qué | Ejemplo en la web |
|---|---|---|
| **Flexbox** | Alinear elementos en una fila (o una columna). | La cabecera: logo a la izquierda, menú a la derecha (`.site-header`). |
| **Grid** | Una cuadrícula de filas y columnas. | Las tarjetas de actividades, los habitantes (`.grid.cols-3`, `.cols-4`). |

```
.cols-3 { grid-template-columns: repeat(auto-fill, minmax(min(100%, 340px), 1fr)); }
← «tantas columnas de al menos 340 px como quepan»: 3 en ordenador, 1 en móvil, sin nada más

@media (max-width: 760px) {                          ← «si la pantalla mide menos de 760 px…»
  .event { grid-template-columns: 1fr; }              ← …los eventos pasan a una sola columna
  .event .thumb { order: -1; }                        ← …y la foto se coloca encima del texto
}
```

Las **media queries** (`@media`) son reglas «si… entonces…» según el tamaño de la pantalla. Pero la cuadrícula «autoadaptable» del primer ejemplo a menudo evita escribirlas: el pie de página funciona igual (`minmax(180px, 1fr)`).

## El menú en el móvil

En pantalla pequeña el menú se pliega tras un botón «Menú». El código está en `src/layouts/Base.astro` (botón `.menu-btn`) y el estilo en `global.css`.

## Los detalles que lo hacen «profesional»

- `text-wrap: balance`: los títulos de dos líneas tienen líneas de longitud similar.
- `@media (prefers-reduced-motion)`: se quitan las animaciones a quien le molestan.
- Un ancho máximo (`--max: 1440px`) para que el texto no se estire en una pantalla gigante.

> **PRUEBA**
>
> F12 → icono de móvil/tableta («modo dispositivo») → elige «iPhone»: mira cómo se reorganiza la web. Ensancha la ventana poco a poco y observa cuándo cambian las columnas.

# 9 · Fotos y vídeos

Las imágenes suelen ser el 80 % del peso de una página. Una foto de móvil pesa de 4 a 8 MB; en la web, la misma foto pesa de 50 a 200 KB. ¿Cómo?

## Tres optimizaciones automáticas

1. **Al subirla en el panel**: la foto se reduce a 2000 px como máximo y se convierte a **WebP**, un formato más ligero que JPEG (ajuste `transformations` en `src/admin/config.yml`).
2. **Al fabricar la web**: Astro crea **5 tamaños** de cada foto (480, 800, 1200, 1600, 1920 px). Archivo: `src/components/Img.astro`.
3. **En el navegador**: gracias a los atributos `srcset` y `sizes`, un móvil descarga la versión pequeña y una pantalla grande la grande. Y `loading="lazy"`: una foto solo se descarga cuando te acercas a ella al desplazarte.

## El texto alternativo

Cada imagen tiene un `alt`: una descripción que se lee a las personas ciegas y que usa Google. P. ej. en el inicio: «Relajación final en la shala del Evergreen Project» (editable en el panel).

## Los vídeos

- **YouTube / Vimeo** (recomendado): se pega el enlace; el componente `Video.astro` solo muestra una miniatura y carga el reproductor al hacer clic (versión «youtube-nocookie», sin cookies) → página rápida y respetuosa.
- **Archivo MP4**: un robot de GitHub (`.github/workflows/compress-videos.yml`) lo recomprime con la herramienta **ffmpeg** (formato H.264, 1280 px máx.).

## El visor a pantalla completa

En `src/pages/[lang]/gallery.astro`, al hacer clic en una foto se abre en grande en un elemento HTML nativo `<dialog>` — sin librería externa: el navegador ya gestiona la ventana, la tecla Esc y el foco.

# 10 · Una web en 8 idiomas

Griego (idioma por defecto), inglés, francés, español, italiano, ruso, alemán, portugués. Tres mecanismos:

1. **Las direcciones** empiezan por el idioma: `/el/`, `/es/`… La carpeta `[lang]` de `src/pages/` genera cada página en cada idioma. La lista de idiomas está en `src/i18n.ts`.
2. **El contenido**: cada archivo YAML tiene una parte por idioma (capítulo 6).
3. **Los textos pequeños** («Menú», «Leer más», «Añadir al calendario»…): `src/content/settings/ui.yml`, 96 frases × 8 idiomas.

## La «reserva» (fallback)

Si una frase no está traducida en un idioma, la web muestra el inglés, y si no, el griego. Nunca hay huecos. El código: función `localize` en `src/lib.ts`.

## Decirle a Google que existen traducciones

En el `<head>` de cada página (`Base.astro`), etiquetas `<link rel="alternate" hreflang="es">` indican las versiones en otros idiomas: un visitante hispanohablante llega a la página en español.

> **MIRAR**
>
> En la web, el selector de idioma arriba a la derecha: mantiene la misma página y solo cambia el prefijo (`/es/contact/` → `/de/contact/`).

# 11 · Las funcionalidades, una a una

Para cada funcionalidad: qué hace, la herramienta o el servicio que usa, y el archivo que hay que abrir.

| Funcionalidad | Cómo está hecha | Dónde mirar |
|---|---|---|
| **Calendario mensual** | Un pequeño script de JavaScript dibuja el mes y colorea los días ocupados; los eventos pasados desaparecen solos (comparando con la fecha de hoy). | `src/pages/[lang]/calendar.astro`, `src/components/EventList.astro` |
| **«Añadir al calendario»** (Google) | Un simple enlace a Google Calendar con el título, las fechas y el huso «Europe/Athens»: Google lo convierte a la hora del visitante. | `src/calendar.ts` (función `googleUrl`) |
| **Apple / Outlook (.ics)** | Un archivo `.ics`: formato de texto estándar (RFC 5545) que entienden todos los calendarios. Las horas se convierten a tiempo universal con la herramienta `Intl` integrada en JavaScript. | `src/calendar.ts` (función `ics`), `src/pages/events/[id].ics.ts` |
| **Suscribirse al calendario** | Un archivo `/calendar.ics` con todos los eventos; el calendario del visitante lo relee con regularidad. | `src/pages/calendar.ics.ts` |
| **Mapa** | Un mapa de **OpenStreetMap** (libre y gratuito) dentro de un `<iframe>`, más un enlace «Abrir en mapas» a Google Maps. | `src/pages/[lang]/[page].astro` (parte de contacto) |
| **Formulario de contacto** | El servicio **Web3Forms**: el formulario envía los campos a su servidor, que los reenvía por e-mail. Una casilla invisible atrapa a los robots («honeypot»). | mismo archivo, `<form action="https://api.web3forms.com/submit">` |
| **Donación / IBAN** | El IBAN en grupos de 4 + un botón que lo copia (API **Clipboard** del navegador). | mismo archivo, bloque `donate` + `<script>` al final |
| **Historias** | Formulario → pequeño servidor (Worker, capítulo 13) → guardada «sin publicar» → un admin la valida. | `src/pages/[lang]/stories.astro`, `worker/index.js` |
| **Galería** | Álbumes en «mosaico» CSS, visor `<dialog>`. | `src/pages/[lang]/gallery.astro` |
| **Banda de aviso** | Si el campo «Ανακοίνωση» del panel está lleno, aparece una banda naranja arriba de todas las páginas. | `Base.astro` (`site.announcement`) |
| **Enlace a los estudios de yoga** | Un enlace externo (sin duplicar: clases y precios siguen en su propia web). | `src/content/settings/site.yml` |
| **Formato de los textos** | Los textos largos se escriben en **Markdown** (`**negrita**`, `- lista`, `[enlace](dirección)`), convertido a HTML por el paquete **marked**. | `src/lib.ts` (función `md`) |

# 12 · El panel de administración (sin código)

Katerina y Kostas no van a editar archivos YAML. Así que les damos una **interfaz** en la dirección `/admin/`: formularios, botones, arrastrar y soltar fotos. Es un **CMS** (Content Management System = sistema de gestión de contenidos).

## El CMS elegido: Sveltia CMS

- Gratuito y de código abierto, funciona por completo en el navegador.
- Lee y escribe directamente los archivos YAML del repositorio: **sin base de datos**.
- Gestiona los 8 idiomas lado a lado y comprime las fotos al subirlas.

## Cómo se configura

Solo dos archivos:

- `public/admin/index.html`: carga el programa Sveltia (versión fija + huella de seguridad, ver capítulo 16).
- `src/admin/config.yml`: describe **cada formulario**. Para cada campo: su nombre, su etiqueta, su tipo («widget»).

```
- { name: start, label: Έναρξη · Start date, widget: datetime, type: date }   ← un calendario para elegir la fecha
- { name: price, label: Τιμή · Price, widget: string, i18n: true }          ← un campo de texto, traducible
- { name: image, label: Φωτογραφία · Photo, widget: image }                ← un botón para subir una foto
- { name: published, label: Published, widget: boolean }                   ← una casilla
```

## Qué pasa al hacer clic en «Save»

Admin: Save → El archivo YAML cambia en el repositorio → Un robot vuelve a fabricar la web → En línea ~2 min después

Cada guardado es un «commit»: siempre se puede volver a una versión anterior.

## La ayuda a la escritura

Junto al campo que se está editando aparece un botón **«✓ Correct»**: propone una versión corregida (ortografía, gramática, puntuación), que se acepta o no. Y en cada guardado, los textos griegos nuevos o modificados se **traducen automáticamente** a los otros 7 idiomas. Ambas cosas pasan por el Worker (capítulo 13), que llama a un servicio de corrección y traducción automáticas. Código: `public/admin/index.html` (el botón) y `worker/ai.js`.

> **MIRAR**
>
> `docs/GUIDE-ADMIN.md`: el manual escrito para las personas administradoras (griego + inglés). Un buen ejemplo de documentación para no técnicos.

# 13 · El pequeño servidor: el Worker

Una web estática no puede guardar secretos ni recibir datos (no hay ningún programa funcionando en el servidor). Así que, para algunas necesidades, añadimos un **servidor diminuto** y gratuito en **Cloudflare**: un **Worker** (una función de JavaScript que se ejecuta cuando se la llama).

| Necesidad | Qué hace el Worker |
|---|---|
| **Acceso de los admins** | Muestra un formulario de e-mail + contraseña, lo comprueba y entrega una «credencial» de sesión (válida 30 días), firmada con una clave secreta (firma HMAC) para que no se pueda falsificar. |
| **Guardados del panel** | Pasa las peticiones del panel al almacenamiento, añadiendo la verdadera clave de acceso que solo conoce el Worker. |
| **Ayuda a la escritura** | Corrige un texto cuando se le pide; traduce cada guardado a los 7 idiomas antes de guardarlo. |
| **Historias** | Recibe el formulario, filtra el spam, comprueba que la foto es realmente una imagen, guarda la historia «sin publicar» y avisa al equipo. |

Los «secretos» (contraseñas de los admins, clave de acceso) se guardan en Cloudflare, cifrados — **nunca** en el código del repositorio.

> **MIRAR**
>
> `worker/index.js` (bien comentado, dividido en secciones: acceso, relevo, historias) · `worker/ai.js` (ayuda a la escritura) · `worker/wrangler.toml` (ajustes) · `worker/README.md` (instalación paso a paso). La herramienta **wrangler** ( `npx wrangler deploy` ) envía el Worker a Cloudflare.

## API: la palabra que hay que conocer

Una **API** es una «puerta» que un programa abre a otros programas: se envía una petición a una dirección y se recibe una respuesta. La web usa varias:

| API | Se usa para |
|---|---|
| API de GitHub | Leer / escribir los archivos de contenido (a través del Worker) |
| Servicio de traducción / corrección | Traducir y corregir los textos del panel (a través del Worker) |
| Web3Forms | Enviar el formulario de contacto por e-mail |
| Cloudflare Turnstile (opcional) | Antirrobots, una alternativa respetuosa a los «CAPTCHA» |
| APIs del navegador | Portapapeles (copiar el IBAN), `Intl` (fechas y husos horarios), `<dialog>` |
| Google Calendar (enlace) | Añadir un evento a tu calendario |

# 14 · Aparecer en Google (SEO)

El SEO (posicionamiento en buscadores) consiste en ayudar a los buscadores a entender y presentar bien la web.

| Elemento | Función | Dónde |
|---|---|---|
| `<title>` y `<meta name="description">` | El título azul y las dos líneas de texto en los resultados de Google. | `Base.astro` |
| Open Graph (`og:image`…) | La vista previa (foto + título) al compartir el enlace en Facebook, WhatsApp… | `Base.astro` |
| Mapa del sitio (`sitemap-index.xml`) | La lista de todas las páginas para Google; generado por **@astrojs/sitemap**. | `astro.config.mjs` |
| `robots.txt` | Indica a los robots qué pueden visitar (el panel no). | `src/pages/robots.txt.ts` |
| Datos estructurados (JSON-LD) | Una ficha legible por máquinas: «esto es una ONG, esta es su dirección, su teléfono» (vocabulario **schema.org**). | `Base.astro` (`ld`) |
| `hreflang` | Las traducciones de cada página. | `Base.astro` |
| Direcciones legibles | `/es/contact/` mejor que `/page?id=37`. | nombres de los archivos en `src/pages/` |

**PRUEBA**

Pega la dirección de la web en *opengraph.xyz* para ver la vista previa al compartir, o busca la «Prueba de resultados enriquecidos» de Google para ver la ficha JSON-LD.

# 15 · Accesibilidad: una web para todo el mundo

Alrededor de una de cada cinco personas tiene una discapacidad (vista, movilidad, audición, atención…). Una web accesible también es más agradable para todos y Google la posiciona mejor.

- **Textos alternativos** en las imágenes (`alt`).
- **Contrastes** suficientes: un texto gris claro sobre fondo crema es ilegible para muchas personas. El crédito del pie de página se oscureció por eso.
- **Títulos en orden** (`h1` y luego `h2`…): los lectores de pantalla los usan como índice.
- **Enlace «Saltar al contenido»** oculto, visible con el teclado (`.skip` en `Base.astro`): evita recorrer todo el menú.
- **Navegación con teclado**: contorno visible en el elemento seleccionado (`:focus-visible` en `global.css`).
- **Nombres claros** para botones y enlaces sin texto (`aria-label`): p. ej. las fotos-enlace del inicio.
- Respeto del ajuste «reducir movimiento» del móvil.

> **PRUEBA**
>
> En la web, pulsa varias veces la tecla **Tab**: siempre debes ver dónde estás. Después F12 → pestaña **Lighthouse** → «Accesibilidad» → Analizar.

# 16 · Seguridad

| Riesgo | Protección en esta web | Dónde |
|---|---|---|
| Que alguien inyecte un script en una página | **CSP** (Content-Security-Policy): una lista blanca de lo que la página puede cargar; el navegador bloquea todo lo demás. | `astro.config.mjs` (`security.csp`) |
| Que un servicio externo (el CMS) se modifique sin que lo sepamos | **Versión fija + SRI**: una huella del archivo; si cambia un solo byte, el navegador se niega a ejecutarlo. | `public/admin/index.html` (`integrity="sha384-…"`) |
| Filtración de contraseñas / claves | Secretos solo en Cloudflare, nunca en el repositorio; archivos de secretos locales ignorados por Git. | `.gitignore`, `worker/README.md` |
| Intentos de contraseña en ráfaga, spam | Límite de 3 intentos / minuto / visitante, campo trampa, tiempo mínimo, Turnstile opcional. | `worker/index.js`, `wrangler.toml` |
| Archivos falsos subidos | Comprobación de los primeros bytes (la «firma»): un JPEG siempre empieza por `FF D8 FF`. | `worker/index.js` (`looksLike`) |
| Espionaje de la conexión | HTTPS en todas partes (el candado). | alojamiento |
| La web mostrada dentro de un marco trampa | Cabecera `X-Frame-Options`. | `public/_headers` |
| Rastreo de los visitantes | Sin cookies, sin herramientas de estadísticas, tipografías y vídeos sin rastreadores. | página de Privacidad |

**PARA DECIR**

«La mejor seguridad es tener poco que proteger: una web estática no tiene base de datos ni contraseñas que robar. Los pocos secretos están guardados en un solo sitio.»

# 17 · Rapidez (rendimiento)

Una web lenta espanta a la gente (y Google la penaliza). Las recetas aplicadas aquí:

- **Páginas ya listas** (estáticas): nada que calcular durante la visita.
- **Imágenes**: WebP, varios tamaños, carga diferida (capítulo 9). La primera foto grande, en cambio, se carga con prioridad (`fetchpriority="high"`).
- **Muy poco JavaScript**: solo pequeños scripts donde hacen falta. Astro no envía JavaScript por defecto.
- **Tipografías alojadas en la web** en lugar de Google Fonts.
- **Vídeos cargados al hacer clic**.
- **Caché**: los archivos cuyo nombre contiene una huella (carpeta `_astro/`) los guarda el navegador durante un año (`public/_headers`).

Resultado medido con **Lighthouse** (integrado en Chrome, pestaña «Lighthouse») en móvil: rendimiento 98–99/100, página de inicio ≈ 256 KB.

## Los indicadores que hay que conocer

| Indicador | Significado | Objetivo |
|---|---|---|
| LCP | Tiempo en mostrar el elemento más grande (a menudo la foto principal) | < 2,5 s |
| CLS | ¿La página «salta» mientras carga? | < 0,1 |
| INP / TBT | ¿La página responde rápido a los clics? | lo más bajo posible |

# 18 · Probar la web

## Las pruebas automáticas

Pequeños programas comprueban que el código hace lo que debe. Aquí con la herramienta integrada en Node (`node:test`), lanzadas con `npm test`:

- `tests/calendar.test.ts`: «las 10 h en Atenas en diciembre = las 8 h en tiempo universal, pero las 7 h en julio (horario de verano)», «el archivo .ics está bien formado»…
- `tests/worker.test.ts`: «una contraseña incorrecta se rechaza», «un archivo de imagen falso se rechaza», «la clave secreta nunca llega al navegador»…
- `tests/ai.test.ts`: «solo se traduce el griego nuevo o modificado», «una traducción escrita a mano se conserva»…

## Las comprobaciones manuales (en cada cambio grande)

| Qué | Cómo |
|---|---|
| Visualización | En un móvil de verdad + el modo dispositivo de las herramientas para desarrolladores; Chrome, Safari, Firefox. |
| Enlaces rotos | Hacer clic en todas partes, o una herramienta de comprobación de enlaces. |
| Accesibilidad | Lighthouse, la extensión **axe DevTools**, navegación con teclado. |
| Errores | F12 → pestaña «Consola»: ninguna línea roja. |
| Contenido | Releer en cada idioma (idealmente personas nativas). |

# 19 · Publicar: alojamiento, dominio, DNS

## El alojamiento

La carpeta `dist/` fabricada por `npm run build` se deja en un **alojamiento** (hosting). Para una web estática bastan servicios gratuitos: **GitHub Pages** (el que se usa hoy), **Cloudflare Pages** (previsto para después), Netlify…

## El robot de publicación

`.github/workflows/deploy.yml` es una receta que GitHub ejecuta automáticamente con cada cambio en `main`: instalar Node → `npm ci` → `npm run build` → publicar `dist/`. Es la **integración / despliegue continuos** (CI/CD): nadie tiene que «subir» nada a mano.

## El nombre de dominio

Un nombre como `evergreenproject.gr` se compra (≈ 10–15 €/año) a un **registrador** (Papaki, DNHOST, OVH…). Se alquila por años, no es tuyo: ¡ojo con la renovación!

## El DNS: la guía telefónica de Internet

El DNS traduce un nombre (`evergreenproject.gr`) en la dirección de un servidor. Se le añaden «registros»:

- **A**: «este nombre apunta a este servidor (dirección IP)».
- **CNAME**: «este nombre es un alias de este otro nombre» (p. ej. `www` → el alojamiento).
- **MX**: «los e-mails de este dominio van aquí».

> **MIRAR**
>
> En `README.md`, las secciones sobre el dominio propio y sobre dejar GitHub: los pasos concretos para conectar un dominio y cambiar de alojamiento.

# 20 · El ciclo de vida de un cambio

## Para quien desarrolla (cambiar el diseño o una funcionalidad)

1. Crear una **rama** (copia de trabajo).
2. Modificar los archivos en VS Code; comprobar en directo con `npm run dev`.
3. Lanzar las pruebas (`npm test`) y la fabricación (`npm run build`).
4. **Commit** con un mensaje claro, **push** a GitHub.
5. Abrir una **pull request**, revisarla y **fusionarla** en `main`.
6. El robot publica: ~2 minutos después, todo el mundo lo ve.

## Para quien administra (cambiar un texto, una fecha, una foto)

1. Abrir `/admin/` e iniciar sesión.
2. Modificar y hacer clic en **Save**.
3. ~2 minutos después está en línea. No hay nada que instalar.

> **EJEMPLO REAL**
>
> En el historial del repositorio («Commits»), busca «Inhabitants: new photo of our dogs»: una foto nueva añadida en `src/assets/uploads/` y una sola línea cambiada en `src/content/inhabitants/dogs.yml` — exactamente lo que hace el panel cuando se cambia una foto.

# 21 · Por dónde empezar a aprender

## Un recorrido recomendado (unos 3 meses a unas horas por semana)

| Etapa | Objetivo | Recursos gratuitos |
|---|---|---|
| 1. HTML | Hacer una página con títulos, textos, imágenes, enlaces | MDN Web Docs («Aprende desarrollo web»), freeCodeCamp |
| 2. CSS | Colores, tipografías, luego Flexbox y Grid | Los juegos **Flexbox Froggy** y **Grid Garden**, web.dev/learn/css |
| 3. Diseño adaptable | Una página que se adapta al móvil | web.dev/learn/design |
| 4. Git y GitHub | Guardar y publicar tu trabajo | GitHub Skills, «Learn Git Branching» |
| 5. JavaScript | Añadir un poco de interacción (botón, menú) | es.javascript.info, freeCodeCamp |
| 6. Un generador | Rehacer una web pequeña con Astro | docs.astro.build (tutorial «Construye un blog») |
| 7. Publicar | Poner tu web en línea gratis | GitHub Pages, Cloudflare Pages |

## Pequeños ejercicios con esta web (sin riesgo, en tu ordenador)

1. Cambiar el color `--clay` en `src/styles/global.css` y observar.
2. Añadir un evento de prueba: copiar un archivo de `src/content/events/`, cambiar las fechas y el título.
3. Cambiar el texto de un botón en `src/content/settings/ui.yml`.
4. Añadir un habitante (¡un animal!) en `src/content/inhabitants/`.
5. Leer `src/components/Story.astro` e intentar crear tu propio componente.

> **CONSEJO FINAL**
>
> «No se aprende leyendo, se aprende rompiendo y arreglando. Con Git nunca se rompe nada de verdad: siempre se puede volver atrás.»

# 22 · Glosario

| | |
|---|---|
| **API** | Puerta de acceso que un servicio abre a otros programas. |
| **Build** | La fabricación de la web final (carpeta `dist/`). |
| **Caché** | Copia que guarda el navegador para no volver a descargar. |
| **CI/CD** | Robots que prueban y publican automáticamente. |
| **CMS** | Interfaz para gestionar el contenido sin programar (aquí Sveltia, en `/admin/`). |
| **Commit** | Un cambio guardado en el historial, con un mensaje. |
| **Componente** | Pieza de página reutilizable (`Img`, `Story`…). |
| **CSP** | Lista blanca de seguridad de lo que una página puede cargar. |
| **DNS** | La guía que traduce un nombre de dominio en la dirección de un servidor. |
| **Repositorio (repo)** | La carpeta del proyecto con todo su historial (en GitHub). |
| **Front-end / back-end** | Lo que se ejecuta en el lado del visitante / en el servidor. |
| **Alojamiento** | Servicio que mantiene los archivos de la web en línea 24 h/24. |
| **i18n** | «Internacionalización» (i + 18 letras + n): gestión de idiomas. |
| **Layout** | El marco común a todas las páginas. |
| **Markdown** | Forma sencilla de escribir texto con formato (`**negrita**`). |
| **npm / paquete** | La tienda de herramientas de JavaScript / una herramienta que se descarga de ella. |
| **Pull request** | Petición de fusionar una rama, con revisión. |
| **Responsive** | Que se adapta a todos los tamaños de pantalla. |
| **SEO** | Posicionamiento: que Google te encuentre y te presente bien. |
| **SRI** | Huella que garantiza que un archivo externo no ha sido modificado. |
| **Estática** | Páginas fabricadas por adelantado y no en cada visita. |
| **URL** | La dirección de una página. |
| **WebP** | Formato de imagen moderno, más ligero que JPEG. |
| **Worker** | Pequeña función de servidor alojada en Cloudflare. |
| **YAML** | Formato de texto legible («clave: valor») que guarda el contenido. |